

Sandbox *orchestration* setup guide

A Kubernetes-based system that spins up isolated React/Vite sandboxes per user, each with its own terminal, live preview, and S3-backed persistence. This guide takes you from a fresh checkout to a working local cluster.

- Kubernetes
- Docker Desktop
- Node.js 20+
- AWS S3
- Scaffold

01 What this project does

A high-level mental model before you touch any code

The orchestrator is an Express API that, on demand, creates a Kubernetes pod containing a user's Vite project, a terminal agent, and a sync sidecar. Each sandbox is reachable at its own subdomain. Files persist in S3, so a sandbox can be torn down and recreated with the same state.

KEY CONCEPT

One pod per sandbox. Each pod runs three containers sharing a single `emptyDir` volume mounted at `/workspace`: the Vite preview, the comm-agent (terminal over socket.io), and the sync-sidecar (watches files and pushes changes to S3).

Per-sandbox URLs

When you start a sandbox you get back two URLs:

URL	ROUTES TO	USED FOR
<code>{id}.preview.localhost</code>	<code>port 5173</code>	Vite dev server output — the live React app the user is editing.
<code>{id}.agent.localhost</code>	<code>port 3000</code>	Comm-agent socket.io — terminal input/output streamed via xterm.js on the frontend.

02

Architecture

How the pieces fit together at runtime

Request flow — starting a new sandbox

- **User clicks "start sandbox" on the frontend**
POST /api/sandbox/start (proxied to the orchestrator)
- **Orchestrator generates a UUIDv7 sandbox ID**
Used as the pod name suffix and S3 prefix
- **Pod created with init container + 3 main containers**
Init: sync-init seeds /workspace from S3 or template, runs npm install, exits
- **Service + Ingress created**
ClusterIP service exposes ports 3000 + 5173, Ingress routes subdomains
- **URLs returned to client**
Frontend opens the preview iframe and connects xterm to the agent over WebSocket

Container responsibilities

CONTAINER	IMAGE	ROLE
sync-init	sandbox-sync	Init container. Downloads files from S3 if the sandbox exists, otherwise copies the bundled template and pushes it to S3. Then runs <code>npm install</code> in <code>/workspace</code> . Exits when done.
sandbox-container	template	Vite dev server bound to 0.0.0.0:5173. Uses <code>npx vite</code> so it picks up node_modules from the shared volume.
sandbox-agent	sandbox-agent	Express + socket.io + node-pty. Spawns an interactive shell in <code>/workspace</code> and pipes I/O over WebSocket. No AWS credentials.
sync-sidecar	sandbox-sync	Runs <code>watch.js</code> . Chokidar watches <code>/workspace</code> and pushes any add/change/delete to S3 instantly. This is the only container with AWS credentials.

WHY THE SPLIT

The user has terminal access to `sandbox-agent`. If AWS credentials lived there, anyone could run `env` and steal them. Putting the credentials in a separate sidecar that the user can never exec into keeps the blast radius zero.

04

Required tools

Install these before doing anything else

TOOL	VERSION	WHY
Docker Desktop	latest	Provides the local Kubernetes cluster. Enable Kubernetes in Docker Desktop settings.
kubectl	≥ 1.28	Bundled with Docker Desktop, but a standalone install is fine.
skaffold	≥ v2.0	Builds all images and applies manifests in one step.
Node.js	≥ 20	For running the frontend. Containers ship their own runtime.
dnsmasq	latest	Wildcard DNS for <code>*.localhost</code> . macOS install via Homebrew shown below.
aws-cli	v2	Optional but useful for inspecting the S3 bucket directly.

Enable Kubernetes in Docker Desktop

Open Docker Desktop → Settings → Kubernetes → check **Enable Kubernetes** → Apply & Restart.
Verify with:

TERMINAL

COPY

```
kubectl config current-context
# Should print: docker-desktop

kubectl get nodes
# Should show one node in Ready state
```

05

Local DNS — wildcard resolver

So `*.preview.localhost` and `*.agent.localhost` resolve to your cluster

Each sandbox lives at a unique subdomain like `019e0d18-bdbd.preview.localhost`. Without a wildcard resolver you'd have to add every sandbox ID to `/etc/hosts` manually — not workable. dnsmasq handles all `*.localhost` hosts in one rule.

TERMINAL

COPY

```
brew install dnsmasq
```

STEP 2 – ADD THE WILDCARD RULE

TERMINAL

COPY

```
echo "address=/.localhost/127.0.0.1" >> $(brew --prefix)/etc/dnsmasq.conf  
  
sudo brew services start dnsmasq
```

STEP 3 – TELL MACOS TO USE DNSMASQ FOR .LOCALHOST

TERMINAL

COPY

```
sudo mkdir -p /etc/resolver  
echo "nameserver 127.0.0.1" | sudo tee /etc/resolver/localhost
```

STEP 4 – VERIFY

TERMINAL

COPY

```
dig +short anything.preview.localhost @127.0.0.1  
# Expected output: 127.0.0.1  
  
ping -c1 random-id.agent.localhost  
# Should reply from 127.0.0.1
```



On Linux, install `dnsmasq` via your package manager and add the same `address=/.localhost/127.0.0.1` line to `/etc/dnsmasq.conf`. Restart NetworkManager or your DNS service afterward.

06

Install Nginx Ingress Controller

Required for subdomain routing — Docker Desktop doesn't include one

TERMINAL

COPY

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-  
  
kubectl wait --namespace ingress-nginx \  
  --for=condition=ready pod \  
  --selector=app.kubernetes.io/component=controller \  
  --timeout=120s
```

VERIFY IT'S BOUND TO LOCALHOST

TERMINAL

COPY

```
kubectl get svc -n ingress-nginx  
# Look for ingress-nginx-controller — EXTERNAL-IP should be "localhost"  
# PORT(S) should include 80:xxxxx/TCP
```



If `EXTERNAL-IP` stays as `<pending>`, your Docker Desktop Kubernetes isn't routing LoadBalancer services to localhost. Restart Docker Desktop and try again. Without this, you'll get `ECONNREFUSED 127.0.0.1:80` when hitting any sandbox URL.

07 Create the S3 bucket

Stores user files across sandbox restarts

STEP 1 — CREATE THE BUCKET

TERMINAL

COPY

```
aws s3api create-bucket \  
  --bucket your-sandbox-bucket-name \br/>  --region ap-southeast-1 \br/>  --create-bucket-configuration LocationConstraint=ap-southeast-1
```



Bucket names must be globally unique. Pick something like `{your-name}-sandbox-storage`. The region must match `AWS_DEFAULT_REGION` set in `pod.js` — currently `ap-southeast-1`.

STEP 2 — BUCKET LAYOUT

Each sandbox gets its own prefix inside the bucket:

```

your-sandbox-bucket-name/
├─ 019e0c88-c124-73c4-.../
│   ├─ package.json
│   ├─ vite.config.js
│   └─ src/...
├─ 019e0d18-bdbd-735a-.../
│   ├─ package.json
│   └─ ...
└─ ...

```

The orchestrator passes `process.env.S3_BUCKET` to each sync sidecar; the sidecar reads from and writes to `{bucket}/{sandboxId}/`.

08 IAM credentials

Scoped to just this bucket — never use root keys



Do not use AWS root credentials. Even though the sync sidecar is locked down, key rotation and audit are far easier with a dedicated IAM user. Treat the access key as production-grade — it goes into a Kubernetes secret your whole team can pull.

STEP 1 — CREATE AN IAM USER

In the AWS console: IAM → Users → Create user → name it `sandbox-sync`. Don't grant console access; programmatic access only.

STEP 2 — ATTACH THIS INLINE POLICY

Replace `your-sandbox-bucket-name` with your actual bucket. The policy grants only what the sync sidecar needs — list, get, put, delete on objects, and list on the bucket itself.

SANDBOX-SYNC-POLICY.JSON

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:PutObject",
        "s3:GetObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ]
    }
  ]
}

```

```

    ],
    "Resource": [
      "arn:aws:s3:::your-sandbox-bucket-name",
      "arn:aws:s3:::your-sandbox-bucket-name/*"
    ]
  }
]
}

```

STEP 3 – GENERATE ACCESS KEYS

Go to the user → Security credentials → Create access key → choose "Application running outside AWS" → save the access key ID and secret somewhere safe. You won't see the secret again.

PERMISSION	WHY THE SYNC AGENT NEEDS IT
<code>s3:ListBucket</code>	To check whether a sandbox prefix already has files (new sandbox vs existing).
<code>s3:GetObject</code>	To download files into <code>/workspace</code> when an existing sandbox is restarted.
<code>s3:PutObject</code>	To seed new sandboxes with the template and to push every file change.
<code>s3:DeleteObject</code>	To mirror file deletions in the watcher.

09 Create the Kubernetes secret

Wrap the AWS keys so they're injected at runtime instead of baked into images

TERMINAL

COPY

```

kubectl create secret generic aws-credentials \
  --from-literal=AWS_ACCESS_KEY_ID=AKIA... \
  --from-literal=AWS_SECRET_ACCESS_KEY=... \
  -n default

```

VERIFY

TERMINAL

COPY

```

kubectl get secret aws-credentials
# NAME          TYPE      DATA   AGE
# aws-credentials  Opaque    2       5s

```


How the orchestrator wires this up

In `sandbox/src/kubernetes/pod.js`, the sync sidecar references the secret via `secretKeyRef`:

POD.JS (SYNC SIDECAR ENV)

```
env: [  
  { name: "AWS_ACCESS_KEY_ID",  
    valueFrom: { secretKeyRef: {  
      name: "aws-credentials",  
      key: "AWS_ACCESS_KEY_ID"  
    }}  
  },  
  { name: "AWS_SECRET_ACCESS_KEY",  
    valueFrom: { secretKeyRef: {  
      name: "aws-credentials",  
      key: "AWS_SECRET_ACCESS_KEY"  
    }}  
  }  
]
```

Set the bucket name in the orchestrator deployment

The orchestrator reads `S3_BUCKET` from its own env at request time and forwards it to the sync sidecar. Edit `k8s/sandbox-deployment.yml`:

K8S/SANDBOX-DEPLOYMENT.YML

```
env:  
- name: S3_BUCKET  
  value: your-sandbox-bucket-name    # ← change this
```



If `S3_BUCKET` is missing, `POST /api/sandbox/start` will return a 500 with "S3_BUCKET env var is not set". The orchestrator deliberately refuses to create a sandbox without it.

10 Build container images

Scaffold handles all four images in one shot

The cluster needs four images locally: `sandbox` (orchestrator), `template` (Vite), `sandbox-agent` (terminal), and `sandbox-sync` (S3 sync). Pod manifests use `imagePullPolicy: Never` so Kubernetes uses the local Docker daemon's images directly.

BUILD EVERYTHING VIA SKAFFOLD

TERMINAL

COPY

```
skaffold build
```

OR BUILD IMAGES INDIVIDUALLY

TERMINAL

COPY

```
# Run from repo root
docker build -t sandbox:latest ./sandbox
docker build -t template:latest ./template
docker build -t sandbox-agent:latest ./sandbox-agents/comm-agent
docker build -t sandbox-sync:latest -f sandbox-agents/sync-agent/dockerfile .
```



The sync image's build context is the repo root (note the trailing `.`) so its dockerfile can `COPY template /template`. Don't run it from inside `sandbox-agents/sync-agent`.

VERIFY ALL FOUR EXIST

TERMINAL

COPY

```
docker images | grep -E "sandbox|template"
# Should list: sandbox, template, sandbox-agent, sandbox-sync
```

11 Deploy to the cluster

Apply the orchestrator deployment, service, ingress, and RBAC

ONE-SHOT DEPLOY WITH SKAFFOLD

TERMINAL

COPY

```
skaffold dev
```

`scaffold dev` builds images, applies all manifests, streams logs, and re-deploys on file changes. Hit `Ctrl+C` to clean up.

OR APPLY MANIFESTS DIRECTLY

TERMINAL

COPY

```
kubectl apply -f k8s/rbac.yaml
kubectl apply -f k8s/sandbox-deployment.yaml
kubectl apply -f k8s/sandbox-service.yaml
kubectl apply -f k8s/ingress.yaml
```

What RBAC does

WHY THIS MATTERS

The orchestrator runs as a pod itself and needs to create pods, services, and ingresses in the `default` namespace. `k8s/rbac.yaml` defines a Role with exactly those verbs and binds it to the `default` ServiceAccount. Without it, every API call the orchestrator makes will return 403.

VERIFY THE ORCHESTRATOR IS RUNNING

TERMINAL

COPY

```
kubectl get pods
# NAME                                READY   STATUS
# sandbox-deployment-xxx-xxx          1/1     Running

curl http://localhost/api/sandbox/health
# {"message": "Sandbox API is healthy"}
```

12 Run the frontend

Lives outside the cluster, proxies API calls to it

TERMINAL

COPY

```
cd frontend
npm install
npm run dev
```

Open `http://localhost:5173` and click **start sandbox**. You should see:

- A new pod appear in `kubectl get pods` within seconds
- Two URLs returned in the response — one for preview, one for the agent
- An xterm terminal mount that streams shell I/O over WebSocket

How the frontend wires to the cluster

The Vite config in `frontend/vite.config.js` proxies `/api` to the orchestrator running inside the cluster:

```
FRONTEND/VITE.CONFIG.JS

server: {
  proxy: {
    '/api': {
      target: 'http://localhost',
      changeOrigin: true,
    }
  }
}
```

The xterm terminal connects directly to `{sandboxId}.agent.localhost` over WebSocket — no Vite proxy involved. dnsmasq + nginx ingress route that traffic to the right pod.

13 Sanity tests

Run these end-to-end after first setup

DNS RESOLVES

TERMINAL

COPY

```
dig +short test.preview.localhost
# 127.0.0.1
```

CLUSTER IS REACHABLE

TERMINAL

COPY

```
curl http://localhost/api/sandbox/health
# {"message": "Sandbox API is healthy"}
```

START A SANDBOX END-TO-END

TERMINAL

COPY

```
curl -X POST http://localhost/api/sandbox/start
# Returns { sandboxId, urls: { previewUrl, agentUrl } }
```

S3 RECEIVED THE SEED

TERMINAL

COPY

```
aws s3 ls s3://your-sandbox-bucket-name/{sandboxId}/
# Should list package.json, vite.config.js, src/, etc.
```

14 Troubleshooting

Real failure modes encountered during development

Symptom	Cause	Fix
ECONNREFUSED 127.0.0.1:80	Nginx ingress not installed or not bound to localhost	Reinstall ingress-nginx; verify EXTERNAL-IP=localhost
503 Service Unavailable	Pod not ready, service selector mismatch, or all containers crashing	<code>kubectl describe pod</code> — check Events
ErrImageNeverPull	Image not built locally	Run <code>scaffold build</code> or build manually
Init:0/1 stuck forever	Sync-init can't reach S3 (bad creds or wrong region)	Check sync-init logs; verify secret + S3_BUCKET
vite: not found	<code>npm install</code> didn't run in <code>/workspace</code>	Check sync-init logs for the install step

Symptom	Cause	Fix
Terminal accepts no input	xterm not focused, or non-interactive shell	Click terminal; verify <code>-i</code> flag in pty spawn
configuration-snippet denied	Cluster admin disabled snippet annotations	Remove snippet; modern nginx handles WS auto

Useful kubectl commands

TERMINAL

COPY

```

# All pods, services, ingresses
kubectl get all

# Why is a pod failing?
kubectl describe pod <pod-name>

# Stream logs from one container
kubectl logs <pod-name> -c <container-name> -f

# Service endpoints — empty means selector mismatch
kubectl get endpoints <service-name>

# Tear down a stuck sandbox manually
kubectl delete pod sandbox-pod-<id>
kubectl delete service sandbox-service-<id>
kubectl delete ingress sandbox-ingress-<id>

```

IF ALL ELSE FAILS

Run `kubectl describe pod <name>` and read the Events section at the bottom. 90% of failures self-explain there: image pull errors, OOM kills, scheduling failures, secret-not-found. Check `kubectl get events --sort-by='.lastTimestamp'` for cluster-level events.

Cluster: docker-desktop · Region: ap-southeast-1 · Namespace: default